

Event Management configuration preferences

- Release version: Australia
-
- Updated March 12, 2026
-
- ⌚ 10 minutes to read

Preferred settings of properties and general configuration.

Use the [Known Error Portal](#) and the [Community](#) to further help you find information issues.

General preferences

Self-health

By default, the self-health monitoring feature is not enabled. To enable it, navigate to Event Management > Settings > Properties and select Yes for the Enable Event Management self-health monitoring (evt_mgmt.self_health_active) property. Use this feature to monitor and track many Event Management features.

Note:

CIs used in the self-health service are created in the CMDB.

Use the following settings to help with preventing performance degradation.

Topic	Details
Business rules	• Avoid writing business rules for event [em_event] tables, as they do not run in the current default REST URL that is used for event injection. • Business rules that are written for alert [em_alert] tables must be highly efficient or they may result in performance degradation. Instead of writing a business rule, consider whether it is more appropriate to write a job. An inefficient business rule can cause incident creation for an alert to fail and the alert impact calculation to fail. • Do not write async business rules for alert tables. • Business rules must not change the Category field on event [em_event] tables.
Scaling up	Check the average event processing time before scaling up event throughput when first starting with Event Management. Do this check after an initial flow of events and all rules are in place. If processing time takes over a few milliseconds per event, determine the cause for the processing slowdown before continuing to scale.

Topic	Details
	Performance duration can be checked in the Performance Statistics [sa_performance_statistics] table.
Configure for large-scale environments	- Set the Enable multi node event processing (evt_mgmt.event_processor_enable_multi_node) property to Yes. Enable multi-node in production environments and set values based on the size of the deployment and expected event rate. - Set the Number of scheduled jobs processing events (evt_mgmt.event_processor_job_count) property to 4. - If you are sending events from a custom source, verify that events have Message Key or Source, Node, Type, and Resource data.
Latency issues for receiving events	Check the following settings: - Verify that the Bucket field in the event [em_event] table is set to a value that is greater than zero (0). - Navigate to System Scheduler > Scheduled Jobs and search for - process events*. Check that all - process events* jobs exist according to the Number of scheduled jobs processing events (event_processor_job_count) property configuration. Verify that the State is Running or Ready. If the state is Queued or Error, set the job state to Ready.
Archive events	- Avoid changing the default retention time for events. - To log events for a longer time, create an archive table and a job that copies new events to it. Do this by scheduling a job to regularly back up events [em_event] to a custom table. - Do not extend table rotation by adding more days.

Event integration

SNMP traps

- Use a monitoring tool to send SNMP traps, rather than sending them directly from devices.
- To avoid having to rewrite event rules, upload MIBs prior to defining the event rules.

Web service API

- Using a web service API for integration can reduce the number of event rules needed. This action avoids having to transform events (prepared data is sent in an event to the instance).

- Use dedicated credentials for integration. Optionally, designate credentials specific to each event source.

CloudWatch

Use dedicated credentials for integrating CloudWatch with ServiceNow.

Email

Use email only if the source has a low volume and other options are not available, such as, running a script or forwarding an SNMP trap.

Event rules

Configuration settings when creating event rules:

- Write Event Rules to apply to the broadest number of events possible. More specific rules can then be created as necessary and should use a lower-order value.
- If a more general rule can achieve the same outcome, avoid writing Event Rules that apply only to a certain subset of events.
- When Event Rules are applied to events, no changes are made to the original event. All processing occurs in memory, so use the Processing Notes field and/or use the Check Process of Event UI action link to troubleshoot.
- If you change a rule/transform that has existing mapping rules, you should review and retest with events that are either actual or simulated.
- Ensure that the From field value exactly matches a string in the JSON in the additional_info field of an event. This matching happens when a rule has been configured based on information in a MIB file. If the MIB file is not uploaded, the JSON for the SNMP trap shows varbinds (variable binding) with dotted names, instead of the translated name in the MIB. The event field mapping rule then fails to be applied.
- Establish a consistent naming convention. A common convention is: <customer acronym>.<Event Source>.<Description>. For example, ACME.OEM.Normalize
- If two Event Rules have similar conditions set, use the Order field to control which Event Rule runs.
- Use Event Rules to associate an alert with a CI.

Additional settings for constructing event rules:

Desired result	Required activity
Effective de-duplication and enabling efficient parallel event processing	Populate the Source, Node, Type, Resource, Metric Name fields.
CI binding	• Bind to host - by populating the Node field and optionally CI identifiers. • Binding to application and / or device – by populating the CI Identifiers field and the Additional information field.

Desired result	Required activity
Alert Correlation, using alert aggregation	Populate the Resource and Metric Name fields. Note: If CI is also bound, Alert Correlation is improved.

Custom event fields

Include additional fields in the Additional information field of the event only.

Do not add additional fields to an event by adding a custom field to the event [em_event] table.

Do not add columns to the event [em_event] table.

For information about how to include additional fields in events, see Custom alert fields.

De-Duplication

Configuration settings for de-duplication.

- The message_key field is used for De-Duplication. If reliable message key values are not provided with the source event, it is important to have a well-defined plan for constructing these identifiers.
- If the message key is not defined, then the default message key is <Source + Node + Type + Resource + Metric Name> .
- The guideline is to have the event source populate the <Source + Node + Type + Resource + Metric Name> fields out-of-the-box and populate the message key. This action enables a better distribution of event processing among instance workers and nodes.
- If the source event does not have values for these fields, make sure to populate them using transform rules. This action does not affect event processing, but is used for de-duplication. Populate as many of these fields as possible before they are sent to the instance. This action provides better distribution of events over the processor workers, and therefore better throughput and scale.

CI binding

- Where possible, always attempt to bind an alert to a CI.

Note:

Although Event Rules are defined on events, CIs are bound to alerts that result from those events, and are not bound to the event.

- To bind a host, machine or any device with an IP, populate the event Node field with a unique host name, FQDN, IP, or MAC address. If other identifiers are necessary to identify a host, then populate the ci_identifiers field with a JSON format. The JSON format must contain the CMDb field name and value to perform the match.

Note:

The event Node field must be populated from an event rule or populated with a unique host name from the source before the event is inserted.

- The primary binding strategy is to use the Node field. If the Node field is not pre-populated in the event, it can be populated using event rules.

Alert settings

Alert lifecycle

General alert functionality:

- An alert is opened whenever an event is not ignored or its threshold is exceeded by an event rule, and de-duplication does not identify the event as belonging to an existing alert.
- An alert is closed when a closing event is sent on the same message key, or the alert is closed manually.
- An alert is reopened if an opening alert that has the same message key is sent within the timeframe defined in properties (default is one hour).
- If an alert is opened and closed at a high rate, as defined in properties, it becomes flapping. When this opening and closing rate stops, the alert goes out of flapping state.
- If an incident is opened from an alert, that alert remains open as long as the incident remains open. By default, when either the incident or the alert is closed the other is closed as well. This behavior can be configured using properties.
- Do not close an alert when creating a corresponding incident.
- Do not delete an open alert. Close an alert first and then delete it.
- Use `Acknowledge` to denote that the alert is known, and can temporarily be ignored.
- Do not use `Acknowledge` to mark an alert as needing attention.
- Do not create alerts in any of these states:
 - `Closed`
 - `OK`
 - `Open`
- The `evt_mgmt.alert_auto_close_interval` property automatically closes alerts after the specified period. Do not specify 0, as this value disables the feature and may lead to performance degradation.
- Do not create alerts in `OK` state. In some monitoring systems `OK` denotes that an issue has been resolved, while in other monitoring systems `OK` is used to denote events that are not of operational significance. For the former case, use `Clear` instead of `OK` using a Mapping Rule. For the latter case, have an Ignore rule, unless the events are of specific value.

Alert action rules

- A scheduled job applies Alert action rules to new Alerts every 11 seconds. If an Alert Rule does not immediately start, allow 10–15 seconds before you start troubleshooting.
- Use the Order field to control which Alert Rule runs if two Alert Rules have similar conditions set.

- Use Alert action rules with Task Templates to populate static values in an incident. Use the populator script to assign dynamic values in the incident. The populator script can return a value of `false` to abort incident creation.
- Create a user called Event Management (or a similar name). Then the Created by field in a task template (for example, Incident) can be set to indicate that user was the source of the task.
- To perform any dynamic value assignment or to override OOB dynamic value assignment, use the `EvtMgmtCustomIncidentPopulator` script include.

Remediation

- Always set orchestration workflow properties to the Remediation Task [em_remediation_task] table.
- Use ECC Queue and Workflow > Live Workflow > All Contexts to find more detailed information on remediation activities.

Business rules

- Business rules created on alert tables should not take more than a few milliseconds. In place of using a business rule, consider if the same functionality can be achieved using a job.
- Do not use business rules to associate an alert with a CI. Use event rules to do binding instead of using business rules.

Planning

- Organize event source configuration of filters, modules, and so on, into multiple parallel efforts, rather than in serial.
- Validate processed event formats to ensure that data that is parsed is aligned with desired results.
- Test production events in a non-production environment. Integrate with non-production element managers and ServiceNow instances. If non-production element managers are not available, send events from element managers to both production and non-production environments.

Services and dashboard

- Use Service Groups to group services into logical groups to reduce the number of services displayed on the Service Health dashboard.
- Import manually built service maps.

Metric Intelligence collector logs and files

Metric Intelligence collector logs and files are located under the path \$(MID_SERVER_DIR)/agent. Use these logs and files for troubleshooting and monitoring purposes.

Table 1. Location of Metric Intelligence collector logs and files

Log or file	Path
PowerShell metric collector log file	Logs/retrieve_metrics{connector instance ID}.log
PowerShell output file	work/metrics/metrics_output_{connector instance ID}.txt
PowerShell input file	work/metrics/parameters_{connector instance ID}.txt

Metric Intelligence performance can be checked in the MID Server log file when the mid.log.level MID Server parameter is in debug mode.

Metric Intelligence performance numbers are available in the Performance Statistics [sa_performance_statistics] table. To view the performance numbers, filter the Performance Statistics list for Metric Collector.

Related concepts

- Rotate event and alert table for cleanup